

Structură proiect laborator

Descriere

Laboratorul la materia CV 3 se va puncta pe baza unui proiect de semestru, ce va include noțiuni prezentate la curs și dezvoltate la laborator pe parcursul întregului semestru. Acesta se va prezenta oral în timpul ultimului curs/laborator. **Studentul are obligația să înțeleagă și să poată explica orice element din cadrul proiectului.** Utilizarea instrumentelor IA este încurajată, cu condiția ca studentul să se documenteze cu privire la tot ce este introdus/modificat de către uneltele ajutătoare.

Alegerea temei

Fiecare student este liber să își aleagă tema dorită. În cazul în care are nevoie de ajutor în selectarea unei teme potrivite, se poate adresa profesorului titular. Singura condiție referitoare la subiectul temei este să aibă legătură cu domeniul de Computer Vision - domeniu foarte vast, care lasă libertate mare de exprimare. Fiecare student va lucra **individual**, deci nu se vor forma echipe pentru acest proiect.

Componenta proiectului

Fiecare proiect va trebui să satisfacă un minim de elemente, prezentat sub forma listei de mai jos:

- Proiectul se va realiza în Python, într-un mediu virtual dedicat (venv, conda, pipenv, poetry etc.) - toate dependențele vor fi instalate în acest mediu virtual. Recomandare pentru utilizarea mediului de dezvoltare [PyCharm](#) sau [VS Code](#) - acestea facilitează crearea întregului lanț de desfășurare a proiectului și integrarea cu multe instrumente utile; de asemenea, studenții universității beneficiază de licență JetBrains prin intermediul contului de email de student.
- Se va alege un framework adecvat pentru dezvoltarea proiectului, e.g. PyTorch, Tensorflow, Keras etc.
- Se va păstra un istoric al dezvoltării proiectului într-un mediu de versionare - recomandare: [GitHub](#) / [GitLab](#). De asemenea, GitHub oferă și varianta desktop, prietenoasă cu utilizatorul: [GitHub Desktop](#).
- Proiectul va trebui să conțină următoarele elemente software:
 - un fișier requirements.txt / environment.yml / pyproject.toml, cu versiunile pachetelor fixate și versiunea de Python menționată;
 - un set de date specific aplicației; acesta nu se va încărca în mediul de versionare;
 - o componentă (script/clasă/funcție) de încărcare a setului de date, care să implice și pre-procesarea setului de date (e.g. Dataset & DataLoader + Transforms). Setul de date se va împărți în cele 3 split-uri necesare: train/val/test, în proporții adecvate proiectului;
 - o componentă (script/clasă/funcție) de creare a unui model de rețea neuronală - alegerea arhitecturii propriu-zisă rămâne la latitudinea studentului, însă acesta trebuie să știe să explice funcționarea fiecărui strat constituent;
 - o componentă (script/clasă/funcție) de antrenare a rețelei neuronale;
 - o componentă (script/clasă/funcție) de testare a rețelei neuronale;
 - o componentă (script/clasă/funcție) de afișare a rezultatelor antrenării + validării + testării (urmărire a evoluției erorilor de antrenare/validare pe parcursul antrenării) sub forma unor grafice;
 - o metodă de salvare a progresului actual (checkpoint, starea optimizatorului, a scheduler-ului, epoca curentă și, ideal, seed-ul sau starea RNG) și încărcare a acestuia pentru reluarea/continuarea antrenării;
 - un fișier README.md cu instrucțiuni de rulare și descărcare a setului de date.
- După ce proiectul este complet funcțional, se va augmenta setul de date **de antrenare** cu o metodă generativă la alegere (VAE, GAN, model de difuzie, etc.) pre-antrenată sau antrenată tot pe setul de date de antrenare și se va reantrena modelul. Seturile de date de validare și testare se vor păstra nealterate. Există posibilitatea ca augmentarea să înrăutățească performanța modelului. Acest lucru este un rezultat valid: se va analiza și se va păstra opțiunea de augmentare.

- Se va întocmi un raport de optimizare a modelului - metricile obținute în funcție de modificarea diferiților hiperparametri pentru a evidenția influența fiecăruia (learning rate, batch size, cu/fără augmentare). Se va păstra un seed fix pentru reproducerea rezultatelor.
- Se va întocmi documentația proiectului - întocmirea diagramei funcționale/pipeline-ului proiectului (reprezentare pe module), se va explica fiecare componentă majoră, codul va fi comentat și se vor explica alegerile făcute. La aceasta se vor adăuga analiza efectelor augmentării și raportul de optimizare. Documentația se va încărca pe Moodle sub format .pdf cu o zi înaintea susținerii proiectului. În documentație se va introduce și o secțiune dedicată utilizării IA care să abordeze sintagma “ce am generat cu IA și cum am verificat”.

Calendar:

- S3: descrierea inițială a proiectului (sarcina, setul de date, metrica, fezabilitatea pe hardware-ul disponibil) - se va încărca pe Moodle în secțiunea dedicată;
- S4: crearea repository-ului pe GitHub/GitLab - link-ul se va încărca pe Moodle în secțiunea dedicată;
- S6: setul de date și DataLoader-ul funcționale - verificabil prin git;
- S9: pregătirea componentei software (fără augmentare, optimizare sau documentație - doar rularea end-to-end a modelului) - verificabil prin git;
- S12: introducerea componentei de augmentare - verificabil prin git;
- S14: prezentarea orală a proiectului, pe calculatorul personal pentru a economisi din timpul necesar descărcării resurselor și instalării pachetelor.

Punctare:

- Descrierea inițială a proiectului: 2 puncte;
- Crearea repository-ului: 2 puncte;
- Commit-uri cu conținut în cel puțin 3 săptămâni distincte (etapele intermediare): $3 \times 2 = 6$ puncte;
- Evaluarea prezentării orale: 30 puncte.

Note:

- Dificultatea temei va fi analizată și se vor propune modificări în cazul în care tema este prea simplă;
- Unele teme pot intra în discordanță cu elementele punctate, e.g. tracking, 3D sau VLM zero-shot nu se încadrează în schema train/val/test + antrenare; acestea vor fi discutate punctual și se va găsi o soluție.